Algoritmul tree

```
chan ch[1:n](int id_transm, in min_id);
chan wakeup[1:n]();
process Proc[p = 1 to n]{
    bool ws = false;
     int wr = 0, r = număr_vecini_p;
    bool Vecini[1:n] = vecinii_lui_p;
     bool rec[1:n] = ([n] * false);
    int V = p;
    enum state{sleep, leader, lost};
     state stare = sleep;
     int id;
    /* aici începe faza wake-up */
    if (p este initiator) {
        ws = true;
        for [q = 1 to r st Vecini[q]]
            wakeup[q]();
    }
```

Algoritmul tree (2)

```
   while (wr < număr_vecini_p){
       receive wakeup[p]();
       wr = wr + 1;
       if (NOT ws) {
           ws = true;
           for [q = 1 to n st Vecini[q]
                 send wakeup[q]();
   }   }

   /* aici începe algoritmul tree */
   while (r > 1){
       receive ch[p](q, id);
       rec[q] = true; r = r - 1;
       V = min(V, id);
   }
```

Dacă este primul wakeup primit ⇒ trimit wakeup la toți vecinii

Aștept să primesc un răspuns de la N – 1 vecini și fac un minim între răspunsurile acestora.

N = numărul de vecini

Algoritmul tree (3)

```
/* de la un singur vecin, q0 nu s-a primit mesaj */
afla q0: Vecini[q0] and NOT rec[q0];
send ch[q0](p, V);
receive ch[p](q0, id);
V = min(V, id);
if (V == p)
    stare = leader
else
    stare = lost

/* informeaza celelalte procese despre decizie */
for [q = 1 to n st Vecini[q] and q<>q0]
    send ch[q](p,V);
```

Aștept rezultatul final de la ultimul nod

primit răspuns